

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0510

Slot A

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Case Study

Code of Conduct:

I AKHILA POCHIMIREDDY (192425225) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.AKHILA

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail.

Analyze the problem and explain how preprocessing enhances edge detection results.

Case Study:

Analysis

In a low-contrast image, the intensity difference between the foreground object and the background is very small. Edge detection algorithms identify edges mainly by detecting sudden changes in intensity. When the contrast is low, these changes become weak, so algorithms such as **Sobel, Prewitt, or Canny** may fail to detect some important edges.

Another problem is that noise can become more noticeable after edge detection. Weak edges may be confused with noise, resulting in missing edges or incorrect edges.

Preprocessing Methods

Before applying edge detection, the image should be enhanced using suitable preprocessing techniques:

1. **Histogram Equalization:** Improves the overall contrast by spreading intensity values over a wider range.
2. **CLAHE:** Enhances local contrast and is particularly useful when different regions of an image have different contrast levels.
3. **Gaussian Filtering:** Reduces noise and prevents unwanted edges from being detected.
4. **Image Normalization:** Adjusts intensity values to a suitable range.
5. **Adaptive Thresholding:** Can be used when illumination varies across different parts of the image.

Solution

A suitable pipeline would be:

Input Image → Contrast Enhancement → Noise Reduction → Canny/Sobel Edge Detection → Final Edge Image

After contrast enhancement, the intensity difference between boundaries becomes stronger. Noise reduction then removes unwanted variations before the edge detector is applied.

Conclusion

Therefore, preprocessing significantly improves edge detection in low-contrast images. **CLAHE or histogram equalization combined with noise filtering** can make weak boundaries more visible and allow the edge detection algorithm to produce clearer and more accurate results.

2. Corner Detection in Textured Regions

A system applies corner detection in highly textured regions, resulting in excessive keypoints. Analyze the issue and suggest methods to improve feature selection.

Case Study:

Analysis

Highly textured regions contain many small intensity variations and patterns. Corner detection algorithms interpret these variations as possible corners, resulting in **excessive keypoints**.

Although detecting many keypoints may appear useful, most of them may be redundant or weak. This increases computational complexity and can make feature matching slower and less reliable.

Problems Caused by Excessive Keypoints

- Increased computational and memory requirements.
- Large number of redundant features.
- Increased matching time.
- Greater possibility of incorrect matches.
- Important features may be difficult to distinguish from weak features.

Methods to Improve Feature Selection

1. Corner Response Threshold:

Set a minimum response value so that only strong corners are selected.

2. **Non-Maximum Suppression:**

When several keypoints occur close together, retain only the strongest point.

3. **Limit Number of Keypoints:**

Select only the top-ranked features based on their corner strength.

4. **Spatial Distribution:**

Ensure that selected keypoints are distributed across the image instead of being concentrated in one textured region.

5. **Feature Ranking:**

Rank detected corners according to their response strength and retain the most distinctive ones.

Solution

The process can be represented as:

Image → Corner Detection → Keypoint Ranking → Thresholding → Non-Maximum Suppression → Selected Keypoints

This produces a smaller set of strong and useful keypoints.

Conclusion

Feature selection is important when working with highly textured regions. Removing weak and redundant corners reduces computational cost while retaining the most useful features for **image representation and matching**.

3. Feature Matching with Noise

Feature matching between two images fails due to noise in one image.

Analyze the impact of noise and explain how preprocessing improves matching accuracy.

Case Study:

Analysis

Feature matching attempts to identify corresponding features between two images. When one image contains noise, the pixel intensity values and local structures are disturbed. This can change the detected keypoints and their descriptors.

As a result, the matching algorithm may produce **incorrect matches, missed matches, or a reduced number of correct correspondences**.

Impact of Noise

Noise can affect the feature-matching process in several ways:

- Changes local pixel intensities.
- Creates false feature points.
- Distorts the shape of existing features.
- Produces different descriptors for corresponding regions.
- Increases the number of incorrect matches.

Preprocessing Methods

1. **Median Filtering:**

Useful for removing impulse or salt-and-pepper noise while preserving edges.

2. **Gaussian Filtering:**

Smooths the image and reduces high-frequency noise.

3. **Image Normalization:**

Makes intensity distributions more consistent between images.

4. **Contrast Enhancement:**

Helps preserve important structures after denoising.

Improved Matching

After preprocessing, feature detection and descriptor extraction should be performed again. A suitable descriptor such as **SIFT or ORB** can then be used for matching.

A robust technique such as **RANSAC** can also be used to reject incorrect matches and retain geometrically consistent correspondences.

Solution

Noisy Image → Denoising → Feature Detection → Descriptor Extraction → Feature Matching → RANSAC → Correct Matches

Conclusion

Noise reduction before feature extraction improves the quality of descriptors and therefore increases matching accuracy. Combining appropriate preprocessing with robust matching techniques provides more reliable image correspondence.

4. PCA and Information Loss

A system applies PCA aggressively and loses important feature details.

Analyze the trade-off between dimensionality reduction and information loss, and justify optimal component selection.

Case Study:

Analysis

Principal Component Analysis (PCA) is a dimensionality-reduction technique used to transform a large number of correlated features into a smaller number of principal components.

The first components contain the highest amount of variance, while later components generally contain less variance. However, if PCA is applied too aggressively and too few components are retained, important feature information may be discarded.

Trade-off Between Reduction and Information Loss

Using fewer components:

- Reduces feature dimensionality.
- Decreases storage requirements.
- Reduces computational complexity.
- May cause significant information loss.

Using more components:

- Preserves more original information.
- Improves representation quality.
- Requires more memory and computation.

- Provides less dimensionality reduction.

Therefore, the goal is not simply to minimize the number of components but to find a suitable balance.

Optimal Component Selection

The **explained variance ratio** can be used to determine how much information is retained by the selected components.

For example, components can be selected until approximately **90–95% of the total variance** is retained, depending on the application. The final selection should also be validated using the performance of the actual computer vision task.

Solution

Original Features → Standardization → PCA → Calculate Explained Variance → Select Components → Reduced Feature Set

Instead of selecting an extremely small number of components, retain enough components to preserve the important characteristics of the original features.

Conclusion

PCA should be applied carefully because excessive dimensionality reduction can remove important image information. Selecting components using explained variance provides a reasonable balance between **computational efficiency and information preservation**.

5. Combined Feature Detection and Matching

A system detects features correctly but fails during matching due to incorrect descriptor selection.

Analyze the issue and explain how appropriate feature descriptors improve matching performance.

Case Study:

Analysis

In this case, the system successfully detects features, but matching fails because the **descriptor selected for representing the features is inappropriate**.

Feature detection identifies important points such as corners or blobs. However, detection alone is not sufficient. Each detected point must be represented using a descriptor that captures its local characteristics. The descriptors from two images are then compared to identify corresponding features.

If the descriptor is not robust to changes such as **rotation, scale, illumination, or noise**, the same feature may produce significantly different descriptors in two images.

Suitable Feature Descriptors

SIFT:

- Robust to scale changes.
- Robust to rotation.
- Provides distinctive descriptors.
- Suitable when accuracy is more important than speed.

ORB:

- Faster than many traditional descriptors.
- Uses binary descriptors.
- Suitable for real-time applications.
- Provides good performance with lower computational cost.

BRIEF:

- Fast binary descriptor.
- Computationally efficient.
- Less robust to rotation unless combined with suitable techniques.

Descriptor Selection

The descriptor should be selected according to the application:

- If images have **scale and rotation changes**, SIFT is a suitable choice.
- If **real-time performance** is required, ORB is a good option.
- If computational resources are limited, a lightweight binary descriptor can be considered.

Improved System

Image → Feature Detection → Keypoint Selection → Descriptor Extraction → Descriptor Matching → Ratio Test/RANSAC → Correct Matches

The descriptor should provide enough distinctive information to differentiate one feature from another.

Conclusion

The failure in the case is not due to feature detection but due to inappropriate descriptor selection. Choosing a descriptor according to the image conditions and application requirements improves the quality of feature representation and produces **more accurate and reliable feature matching**.